CSE 220 Online Exam Problem Bank

DFT/FFT Applications with Full Solutions

Exam Preparation Guide

Contents

1 Introduction

This document contains 6 exam-style problems that apply the offline DFT/FFT code (`transforms.py`) to solve real problems. Each problem includes:

- Problem statement
- Core concept explanation
- Line-by-line Python solution using your offline code
- Key viva questions

Prerequisite: Your `transforms.py` file must be available and importable. All solutions use:

```
from transforms import FFTTransformer, next_power_of_two
```

2 The Three Frequency-Domain Operations

Before jumping into problems, memorize these three formulas — every problem uses one of them:

Core Frequency-Domain Operations

Operation	Frequency Formula	Used For
Convolution	`IFFT( FFT(A) * FFT(B) )`	Multiplying, blurring
Cross-Correlation	`IFFT( FFT(A) * conj(FFT(B)) )`	Finding shifts
Deconvolution	`IFFT( FFT(A) / FFT(B) )`	Undoing convolution (decrypt/unblur)

Golden Rules

- **Convolution vs Cross-Correlation:** Cross-correlation uses `np.conj()` on one spectrum.
- **Deconvolution:** Always replace near-zero frequencies with `1e-12` to avoid division-by-zero.
- **Linear convolution:** Zero-pad both inputs to length $\geq n + q - 1$.
- **Circular convolution:** No padding needed; work at the exact input length.

3 Problem 1: Large Integer Multiplication (Digit Arrays LSD-first)

3.1 Problem Statement

Given two non-negative integers as arrays of base-10 digits (LSD-first), compute their product using FFT-based convolution.

Example:

- Input: $A = [3, 2, 1]$ (represents 123), $B = [5, 4]$ (represents 45)
- Output: $[5, 3, 5, 5]$ (represents $123 \times 45 = 5535$)

3.2 Concept

Why Convolution?

Numbers written in base 10 are polynomials evaluated at $x = 10$:

$$123 = 3 + 2x + 1x^2, \quad 45 = 5 + 4x$$

Multiplying two numbers = multiplying two polynomials = convolving their coefficient arrays.

Steps:

1. Zero-pad both arrays to $N =$ `next_power_of_two`$(m + n - 1)$.
2. Perform FFT convolution: `IFFT(FFT(A) * FFT(B))`.
3. **Carry sweep:** Each output slot must be a single digit 0–9. Loop through and propagate carries using `% 10` and `// 10`.

3.3 Solution

```python
import numpy as np
from transforms import FFTTransformer, next_power_of_two

def multiply_digit_arrays(A, B):
    min_length = len(A) + len(B) - 1
    N = next_power_of_two(min_length)

    A_pad = np.zeros(N, dtype=np.complex128)
    B_pad = np.zeros(N, dtype=np.complex128)
    A_pad[:len(A)] = A
    B_pad[:len(B)] = B

    engine = FFTTransformer()
    F_C = engine.transform(A_pad) * engine.transform(B_pad)
    C = np.round(engine.inverse(F_C).real).astype(np.int64)

    # Carry sweep (base 10)
    result = []
    carry = 0
    for i in range(min_length):
        total = C[i] + carry
        result.append(total % 10)
        carry = total // 10
```

```
    while carry > 0:
        result.append(carry % 10)
        carry //= 10
    while len(result) > 1 and result[-1] == 0:
        result.pop()
    return result

# Test
print(multiply_digit_arrays([3, 2, 1], [5, 4]))    # [5, 3, 5, 5]
print(multiply_digit_arrays([9, 9, 9], [9, 9]))    # [1, 0, 9, 8, 9]
```

3.4 Carry Sweep Explained

Raw convolution output: $[15, 22, 13, 4]$ (these aren't digits yet!).

Step	Total	Keep (% 10)	Carry (// 10)	Result so far
i=0	$15 + 0 = 15$	5	1	[5]
i=1	$22 + 1 = 23$	3	2	[5, 3]
i=2	$13 + 2 = 15$	5	1	[5, 3, 5]
i=3	$4 + 1 = 5$	5	0	[5, 3, 5, 5]

Final: $[5, 3, 5, 5] = 5535$ ✓

4 Problem 2: Weighted Polynomial Product

4.1 Problem Statement

Given polynomials $P(x)$ and $Q(x)$ (MSD-first) and a weight sequence W:

$$R[k] = \sum_{i=0}^{k} w_i \cdot p_i \cdot q_{k-i}$$

Example: $P = [1, 3, 2]$, $Q = [4, 1]$, $W = [3, 2, 1] \Rightarrow R = [12, 27, 14, 2]$

4.2 Concept

Key Insight

The coefficient formula $R[k] = \sum w_i p_i q_{k-i}$ is the convolution of **(P weighted by W)** with **Q**. Just multiply $P \cdot W$ element-wise first, then convolve with Q.

Steps:

1. Reverse arrays to LSD-first (so index i = power x^i).
2. Compute weighted P: `pw = p * w`.
3. FFT-convolve `pw` with `q`.
4. Reverse result back to descending-power format.

4.3 Solution

```python
import numpy as np
from transforms import FFTTransformer, next_power_of_two

def weighted_poly_mult(P, Q, W):
    p = np.array(P)[::-1]  # reverse to LSD-first
    q = np.array(Q)[::-1]
    w = np.array(W)[::-1]

    pw = p * w  # multiply P by weights

    min_length = len(p) + len(q) - 1
    N = next_power_of_two(min_length)

    pw_pad = np.zeros(N, dtype=np.complex128)
    q_pad = np.zeros(N, dtype=np.complex128)
    pw_pad[:len(pw)] = pw
    q_pad[:len(q)] = q

    engine = FFTTransformer()
    F_R = engine.transform(pw_pad) * engine.transform(q_pad)
    R = np.round(engine.inverse(F_R).real).astype(np.int64)[:min_length]

    return R[::-1].tolist()

# Test
print(weighted_poly_mult([1,3,2], [4,1], [3,2,1]))  # [12, 27, 14, 2]
```

5 Problem 3: Rolling Shutter Correction (Per-Row Shift)

5.1 Problem Statement

Due to a rolling shutter effect, each row of an image is horizontally shifted by a different random amount. Detect and reverse each row's shift.

5.2 Concept

Cross-Correlation for Shift Detection

Sliding one signal over another and measuring similarity is **cross-correlation**. The index of the maximum value is the shift amount.

$$\text{Corr}(A, B) = \text{IFFT}\big(\text{FFT}(B) \cdot \overline{\text{FFT}(A)}\big)$$

Why `np.conj()`? Multiplying spectra performs convolution (flips signal). Conjugating one spectrum unflips it, giving cross-correlation.

5.3 Solution

```python
import numpy as np
from transforms import FFTTransformer

def fix_rolling_shutter(original_img, shifted_img):
    orig = np.asarray(original_img, dtype=np.float64)
    shift = np.asarray(shifted_img, dtype=np.float64)
    H, W = orig.shape
    engine = FFTTransformer()

    fixed_img = np.zeros_like(orig)
    for r in range(H):
        F_orig = engine.transform(orig[r, :])
        F_shift = engine.transform(shift[r, :])
        cross_corr = engine.inverse(F_shift * np.conj(F_orig)).real
        shift_amount = int(np.argmax(cross_corr))
        fixed_img[r, :] = np.roll(shift[r, :], -shift_amount)

    return fixed_img
```

Why `np.roll(row, -shift_amount)`?

Cross-correlation tells us the row was shifted **right** by `shift_amount`. Rolling with a **negative** value shifts **left** to undo the distortion.

6 Problem 4: BUET Logo Decryption (Deconvolution)

6.1 Problem Statement

An alien picked one row (the "key row") and **circularly convolved** it with all other rows. The key row itself was left unchanged and has **small pixel values**. Decrypt the image.

6.2 Concept

Deconvolution

If blurring/encryption is:

$$\text{Encrypted} = \text{Original} \circledast \text{Key}$$

Then:

$$\text{FFT}(\text{Encrypted}) = \text{FFT}(\text{Original}) \cdot \text{FFT}(\text{Key})$$

$$\Rightarrow \text{Original} = \text{IFFT}\left(\frac{\text{FFT}(\text{Encrypted})}{\text{FFT}(\text{Key})}\right)$$

Steps:

1. Find the key row: use `np.argmin` on column sums.
2. Compute FFT of the key (once).
3. Deconvolve every other row via division in the frequency domain.

6.3 Solution

```python
import numpy as np
from transforms import FFTTransformer

def decrypt_buet_logo(encrypted_img):
    img = np.asarray(encrypted_img, dtype=np.float64)
    H, W = img.shape
    engine = FFTTransformer()

    # Find the key row (smallest sum)
    row_sums = np.sum(np.abs(img), axis=1)
    key_idx = int(np.argmin(row_sums))
    key_row = img[key_idx, :]

    # FFT of key (once)
    F_key = engine.transform(key_row)
    F_key_safe = F_key.copy()
    F_key_safe[np.abs(F_key_safe) < 1e-12] = 1e-12

    decrypted = np.zeros_like(img)
    for r in range(H):
        if r == key_idx:
            decrypted[r, :] = key_row
        else:
            F_enc = engine.transform(img[r, :])
            F_orig = F_enc / F_key_safe
            decrypted[r, :] = engine.inverse(F_orig).real

    return np.clip(decrypted, 0, 255), key_idx
```

Division-by-Zero Safety

Always replace near-zero frequencies with `1e-12` to prevent numerical explosion:

```
F_key_safe[np.abs(F_key_safe) < 1e-12] = 1e-12
```

7 Problem 5: Satellite Image 2D Shift Correction

7.1 Problem Statement

A satellite image has shifted by an unknown amount horizontally AND vertically (both shifts affect the whole image uniformly). Detect and reverse.

7.2 Concept

Row & Column Cross-Correlation

Because the shift is the same for all rows/columns, we only need:

- **One row** to detect horizontal shift.
- **One column** to detect vertical shift.

Pick the row/column with the highest variance (most distinct content) to get the most reliable peak.

7.3 Solution

```python
import numpy as np
from transforms import FFTTransformer

def cross_correlate_1d(a, b, engine):
    F_a = engine.transform(a.astype(np.complex128))
    F_b = engine.transform(b.astype(np.complex128))
    corr = engine.inverse(F_b * np.conj(F_a)).real
    return int(np.argmax(corr))

def realign_satellite(original_img, shifted_img):
    orig = np.asarray(original_img, dtype=np.float64)
    shift = np.asarray(shifted_img, dtype=np.float64)
    engine = FFTTransformer()

    # Best row (max variance) for horizontal shift
    best_row = int(np.argmax(np.var(orig, axis=1)))
    h_shift = cross_correlate_1d(orig[best_row], shift[best_row], engine)

    # Best column for vertical shift
    best_col = int(np.argmax(np.var(orig, axis=0)))
    v_shift = cross_correlate_1d(orig[:, best_col], shift[:, best_col], engine)

    fixed = np.roll(shift, -h_shift, axis=1)
    fixed = np.roll(fixed, -v_shift, axis=0)
    return fixed, h_shift, v_shift
```

Rolling Shutter vs Satellite Shift

- **Rolling Shutter:** Each row has a *different* shift. Loop over all rows.
- **Satellite:** All rows have the *same* horizontal shift, all columns share the *same* vertical shift. Only need 1 row + 1 column.

8 Problem 6: Alien Multiplication (Large Numbers as Strings)

8.1 Problem Statement

Given two huge integers as decimal strings, compute their product using FFT-based multiplication.

Test Case: $65767879797907 \times 765454532435435345 = 50342321679976816694244164822915$

8.2 Concept

Same as Problem 1, but input is a number/string in MSD-first order. Just convert to a digit array, reverse to LSD-first, run the same pipeline.

8.3 Solution

```python
import numpy as np
from transforms import FFTTransformer, next_power_of_two

def alien_multiply(num1, num2):
    A = [int(d) for d in str(num1)][::-1]  # LSD-first
    B = [int(d) for d in str(num2)][::-1]

    min_length = len(A) + len(B) - 1
    N = next_power_of_two(min_length)

    A_pad = np.zeros(N, dtype=np.complex128)
    B_pad = np.zeros(N, dtype=np.complex128)
    A_pad[:len(A)] = A
    B_pad[:len(B)] = B

    engine = FFTTransformer()
    F_C = engine.transform(A_pad) * engine.transform(B_pad)
    C = np.round(engine.inverse(F_C).real).astype(np.int64)

    result = []
    carry = 0
    for i in range(min_length):
        total = C[i] + carry
        result.append(total % 10)
        carry = total // 10
    while carry > 0:
        result.append(carry % 10)
        carry //= 10
    while len(result) > 1 and result[-1] == 0:
        result.pop()

    return int(''.join(str(d) for d in result[::-1]))

# Test
print(alien_multiply(65767879797907, 765454532435435345))
# 50342321679976816694244164822915
```

9 Bonus: General Blurred Image Recovery (Kernel Known)

9.1 Problem Statement

Given a blurred image and the blur kernel, recover the original image.

9.2 Concept

2D Deconvolution

Same principle as Problem 4 but in 2D:

$$\text{Original} = \text{IFFT2D}\left(\frac{\text{FFT2D(Blurred)}}{\text{FFT2D(Kernel)}}\right)$$

9.3 Solution

```python
import numpy as np
from transforms import FFTTransformer

def fft2d(plane, engine):
    P, Q = plane.shape
    rows = np.zeros_like(plane, dtype=np.complex128)
    for r in range(P):
        rows[r, :] = engine.transform(plane[r, :])
    result = np.zeros_like(plane, dtype=np.complex128)
    for c in range(Q):
        result[:, c] = engine.transform(rows[:, c])
    return result

def ifft2d(plane, engine):
    P, Q = plane.shape
    rows = np.zeros_like(plane, dtype=np.complex128)
    for r in range(P):
        rows[r, :] = engine.inverse(plane[r, :])
    result = np.zeros_like(plane, dtype=np.complex128)
    for c in range(Q):
        result[:, c] = engine.inverse(rows[:, c])
    return result

def unblur_image(blurred, kernel):
    img = np.asarray(blurred, dtype=np.float64)
    kern = np.asarray(kernel, dtype=np.float64)
    engine = FFTTransformer()
    H, W = img.shape
    kh, kw = kern.shape

    kern_pad = np.zeros((H, W), dtype=np.float64)
    kern_pad[:kh, :kw] = kern

    F_blur = fft2d(img.astype(np.complex128), engine)
    F_kern = fft2d(kern_pad.astype(np.complex128), engine)

    F_kern_safe = F_kern.copy()
    F_kern_safe[np.abs(F_kern_safe) < 1e-12] = 1e-12

    F_orig = F_blur / F_kern_safe
    original = ifft2d(F_orig, engine).real
    return np.clip(original, 0, 255)
```

10 Problem-to-Technique Mapping (Quick Reference)

Which Technique for Which Problem?

Problem Type	Technique + Formula
Multiply large numbers / polynomials	Convolution + Carry sweep `IFFT(FFT(A) * FFT(B))`
Detect image/signal shift	Cross-correlation `IFFT(FFT(B) * conj(FFT(A)))`
Undo blur (kernel known)	Deconvolution `IFFT(FFT(Blurred) / FFT(Kernel))`
Decrypt convolved image (key hidden)	Find key + Deconvolution Key = row with smallest sum
Apply blur / low-pass filter	Convolution (Task B offline)
Detect rotation / scale changes	(Advanced, uncommon) log-polar FFT

11 Exam Strategy

5-Step Approach for Any DFT Problem

1. **Identify the operation:** Multiplication? Shift detection? Blur/unblur?
2. **Pick the formula:** Convolution / Cross-correlation / Deconvolution.
3. **Handle padding:** Linear needs $n + q - 1$ padded to power of 2. Circular needs no padding.
4. **Apply carry sweep / `argmax` / cropping** as required for the domain.
5. **Guard division:** If deconvolving, add `1e-12` safety.

Common Debugging

- Wrong shift detected → Check if you conjugated the correct spectrum.
- Big-int product off by ×10 → Missing `zfill(base_digits)` or wrong LSD/MSD order.
- Deconvolution result explodes → Missing `1e-12` safety.
- Image shifted diagonally → Forgot to crop `[kh//2:kh//2+H, kw//2:kw//2+W]`.
- Blur shows edge wraparound → Used circular instead of linear convolution.